HONEYWELL EMBEDDED ENGINEERING TEAMS — PA · IA · BA

AI Champions Programme

Module 06

Embedded Test Strategy, Validation and Regression Engineering

Proving what Module 05 built actually works — host-based tests, mocks for hardware, static analysis, and a defect the strategy has to catch.

DURATION

2.5 Hours · Module 6 of 12

FORMAT

Guided Build + Break-It Lab

DELIVERED FOR

Honeywell Embedded Engineering

NIIT StackRoute × RPS Programme Design

How We'll Spend These Two and a Half Hours

Six connected moves — from a spec-derived strategy to proof that it actually catches defects.

01

Spec-to-Test Traceability

20 min

Derive a test strategy directly from the Module 04/05 specification and acceptance criteria.

02

The Embedded Test Pyramid

20 min

Unit, integration & software-level E2E — what belongs at each layer, and why.

03

Mocks, Stubs & Fakes

25 min

Test hardware-dependent interfaces without hardware in the loop.

04

Static Analysis & Build Validation

20 min

The automated gate that runs before any human looks at the diff.

05

Write & Run the Test Suite

35 min

Create/update unit and integration tests for the Module 05 feature.

06

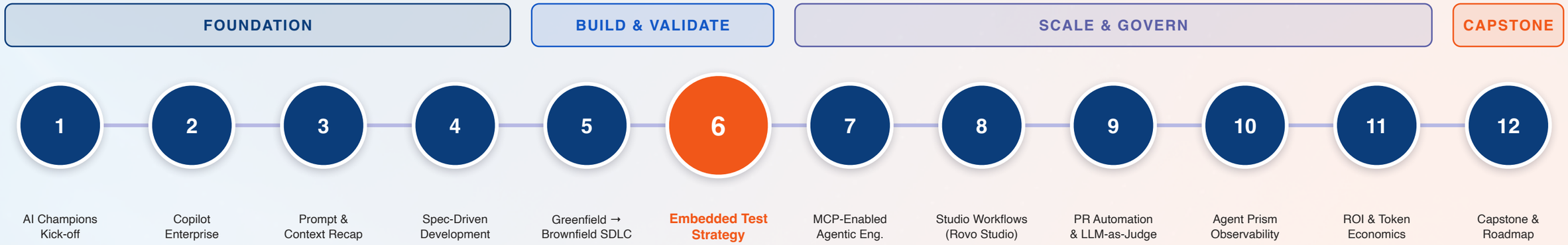
Inject a Defect, Prove the Strategy

30 min

Break it on purpose — then prove the test strategy catches it.

Module 06 in the 12-Module Journey

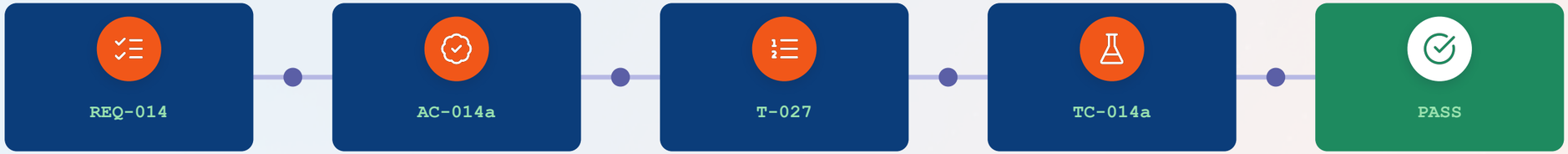
Where Modules 04 and 05 get proven — a spec is only as good as its ability to catch a real defect.



Why this matters: Module 09's PR quality gates check for test evidence exactly like today's. Module 07's MCP workflows automate running these same checks. The traceability chain from Module 04 gets its final link here: requirement → test.

Spec-to-Test Traceability: Closing the Loop

Module 04's chain gets its final, provable link — every acceptance criterion becomes an executable test.



The test strategy question: For every acceptance criterion, ask “what test proves this?” If you can't name one, the strategy has a gap — not the code.

What Counts as “Derived From the Spec”

A strategy is spec-derived when every acceptance criterion maps to at least one named test, every stated constraint has a corresponding boundary or negative test, and every test case's purpose is traceable back to a requirement ID — not invented after the fact to pad coverage numbers.

The Embedded Test Pyramid

Three layers, growing cost and confidence — add the next one only when the last isn't enough.

Host-Based Unit Tests

Fast, isolated tests for individual functions — run on your dev machine, no hardware needed.

Seconds per run

Integration Tests

Verify modules work together correctly — real interfaces, mocked hardware boundaries.

Minutes per run

Software-Level E2E Tests

Full feature validation against acceptance criteria — closest to production behaviour, without HIL.

Longest, highest confidence

Hardware-in-the-loop: Discussed as a concept above the E2E layer for teams that need it — not demonstrated in this programme, which stays entirely software-level.

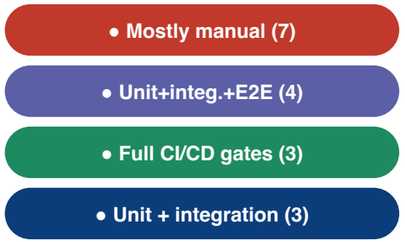
What the Calibration Survey Told Us About Testing Today

Real answers from 19 Honeywell engineers — the automation gap this module exists to close.

What Does Your Test Pipeline Look Like Today?

7

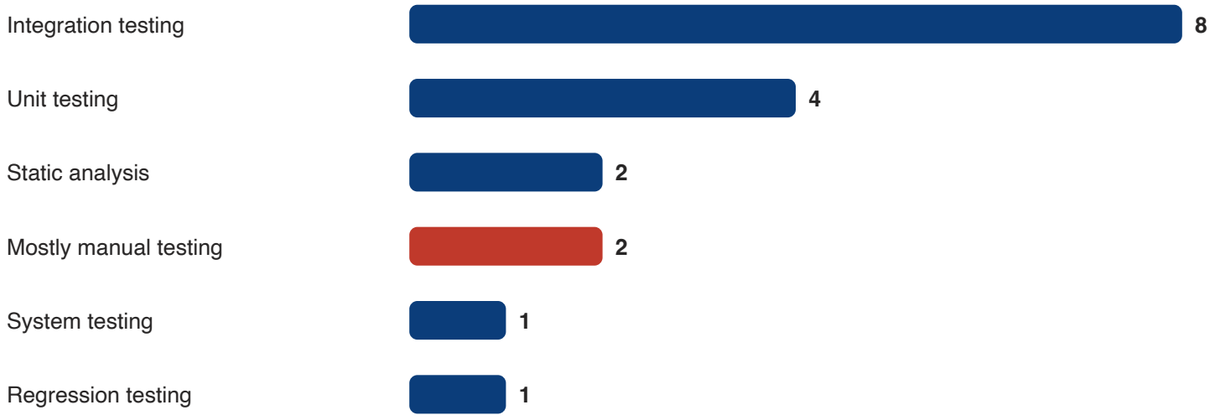
of 19



Only 3 of 19 have automated tests wired into CI/CD with quality gates. “Mostly manual” is the single most common answer.

Before code reaches review today: 8 of 19 say “very little” or “build and compile only” runs automatically — the static analysis gate in this module changes that.

Which Testing Practices Are Common on Your Projects?



The gap this module closes: Only 1 of 19 explicitly named regression testing as a common practice — today's “inject a defect” lab makes it concrete for everyone.

Mocks, Stubs & Fakes: Testing Without Hardware

Three techniques, one goal — exercise your logic without a physical board on the bench.

Stub

Returns canned data, nothing more

Replace a sensor-read function with one that always returns a fixed value — enough to test the logic that consumes it.

Mock

Verifies how it was called

Replace a driver call with one that records whether — and how — it was invoked, then assert on that behaviour.

Fake

A working, simplified substitute

A lightweight in-memory implementation of flash storage — real behaviour, without the real hardware constraints.

Boundary & Negative Testing

The tests most likely to get skipped under deadline — and most likely to catch a real embedded defect.

BOUNDARY TESTING

- Test at the exact edge of a valid range — not just comfortably inside it.
- Buffer at exactly its maximum size, not one byte under.
- Counter at its rollover value — 0xFFFF wrapping to 0x0000.
- Timing constraint at exactly the stated threshold, not a safe margin away.

NEGATIVE TESTING

- Deliberately feed invalid input and confirm the system fails safely.
- Null pointer, malformed packet, or out-of-range command byte.
- Sensor read timeout — does the system degrade gracefully or hang?
- Two interrupts firing back to back — does state stay consistent?

Connect it back: This is Module 04's “edge & negative cases” acceptance-criteria check, made concrete as actual test code.

Static Analysis, Build Validation & Sanitizers

The automated gate that runs before any human opens the diff.

Static Analysis

Catches coding-standard violations, unreachable code & likely bugs — no execution required.

Build Validation

Confirms clean compilation across every target in the toolchain, not just one.

Sanitizers (Where Applicable)

Runtime checks for memory errors, undefined behaviour & data races — on host builds.

Feeds the PR Directly

These same checks reappear as evidence in Module 09's PR quality gate.

Run this before you ask for review: A PR that already passed these four checks gets reviewed for judgment, not caught on mechanics.

Regression Testing: Proving Nothing Broke

The check brownfield work from Module 05 depends on most — and the one most often skipped.

WITHOUT REGRESSION TESTING

- A brownfield change “works” by only checking the new behaviour.
- Existing callers of the changed interface are assumed safe, not verified.
- A regression surfaces in the field, weeks after the change shipped.

WITH REGRESSION TESTING

- The full existing test suite runs against every brownfield change, not just new tests.
- A failing existing test is the strategy working — caught before merge, not after.
- Behaviour preservation from Module 05's checklist becomes provable, not assumed.

Hands-On Lab: Derive, Build, Break, Prove

Four stages, one deliberate defect — the strategy is only real once it catches something.

Derive

Pull the test strategy directly from the Module 05 feature's spec & acceptance criteria.

Build

Create/update unit & integration tests; mock hardware-dependent interfaces where needed.

Break

Inject a deliberate defect into the implementation — something a real review might miss.

Prove

Run the full suite. If it doesn't catch the defect, the strategy — not the code — needs work.

The real lesson: If the defect slips through, that's not a failed lab — it's the most valuable finding of the afternoon.

Test Evidence: What “Proven” Actually Looks Like

Not “trust me, it works” — evidence a reviewer can check in under a minute.

Traceability Table

Every acceptance criterion listed next to the test ID(s) that prove it.

Build & Static Analysis Log

Clean output from the toolchain and static analyzer, attached or linked.

Test Run Output

Pass/fail results for the full suite — not a screenshot of the ones that passed.

Regression Confirmation

Explicit note that the full existing suite ran and passed, not just new tests.

Where this goes next: This exact evidence package is what Module 09's LLM-as-Judge checks for automatically before a human ever looks.

Anti-Patterns: How Test Strategies Fail in Practice

Five habits that quietly turn “we have tests” into false confidence.

DO THIS	NOT THAT
Derive tests from acceptance criteria, before implementation finishes	Write tests after the fact to match whatever the code already does
Run the full regression suite on every brownfield change	Run only the new tests and assume the rest still passes
Mock hardware-dependent interfaces to test logic in isolation	Skip tests that touch hardware because “we can't test that here”
Treat a caught defect in the lab as proof the strategy works	Treat an uncaught defect as a reason to skip testing next time
Attach real test evidence — logs, traceability, pass/fail output	Say “it's tested” with nothing a reviewer can actually check

Module 06 Outcomes & What's Next

Everything below should be true before we connect agents to real engineering tools in Module 07.

- Can derive a test strategy directly from a specification's acceptance criteria
- Can write mocks, stubs & fakes for hardware-dependent interfaces
- Understand static analysis, build validation & sanitizers as a pre-review gate
- Injected a real defect and proved the test strategy caught it
- Know the three test-pyramid layers and when each one is enough
- Practiced boundary and negative testing on real embedded scenarios
- Ran the full regression suite against a brownfield change
- Know what test evidence looks like — and why Module 09 checks for it

NEXT — MODULE 07: MCP-ENABLED AGENTIC EMBEDDED ENGINEERING AND REUSABLE SKILLS

2 hours · Connect agents to approved repositories, build/test tools & CI services — then package today's work as a reusable, versioned skill.

Questions & Discussion

Module 06 — Embedded Test Strategy, Validation and Regression Engineering